

LAB1: Experimental Environment Configuration

Linux, C++, and Git Fundamentals

Li Yunzhe

Student ID: 24020036038

25 September 2026

Abstract

This laboratory established the basic development environment required for later visual-navigation work and verified fundamental Linux shell, CMake, C++, and Git skills. The shell exercises produced 19,567 total lines, 97,676 words, and 14,338 non-empty lines from the supplied text. A small CMake project successfully built a library and linked executable. The main C++ task implemented a `RandomVector` class with printing, mean, minimum, maximum, and histogram functions. A deterministic test containing 20 values returned a mean of 0.55804, a minimum of 0.0278694, and a maximum of 0.982655. Version-control practice covered repository creation, staging, commits, restoration, and synchronization with a remote repository. All required programs compiled and executed successfully.

1 Introduction

LAB1 introduces the software tools used throughout the Visual Navigation course. Its purpose is not to develop a complete robotics algorithm, but to establish a reliable workflow before more complex laboratories are attempted. The work follows the course LAB1 material and the MIT Visual Navigation laboratory structure [1]. It covers Linux file and text processing, basic C++ language behaviour, class implementation, compilation, and distributed version control.

This was completed as an individual laboratory. The report records the actual environment, implementation choices, observed outputs, and problems encountered. It does not assume results that were not measured during the experiment.

2 Objectives

The objectives were to:

- configure a usable Ubuntu-based development environment;
- practise file navigation, redirection, pipelines, searching, and text statistics;
- configure and build a small C++ library and executable with CMake;
- understand selected C++ operators, references, pointers, arrays, and numeric conversions;
- implement and test the required `RandomVector` class;
- practise the Git workflow from local modification to remote synchronization; and
- verify the results in a reproducible, version-controlled workspace.

3 Experimental Environment

The experiment was performed in Ubuntu 22.04.5 LTS running under WSL2. Table 1 lists the verified software versions. The environment supported all required compilation, shell, and Git operations.

Table 1: Verified experimental environment.

Component	Verified configuration
Linux environment	Ubuntu 22.04.5 LTS under WSL2
C++ compiler	GNU g++ 11.4.0
Build configuration tool	CMake 3.22.1
Build tool	GNU Make 4.3
Version control	Git 2.34.1
C++ language mode	C++11 with warning and standards-conformance checks

4 Methods

4.1 Linux shell practice

The shell work began with directory navigation and inspection, followed by file creation, copying, renaming, searching, and safe deletion. Redirection was tested in both overwrite and append modes. Pipelines were then used to connect text-processing tools so that the output of one operation became the input of the next. The supplied Dante text was analysed for total lines, words, and non-empty lines. Five independent messages generated by the `fortune` utility were also collected into a single text file.

4.2 Git workflow

Git practice was performed in a dedicated repository. A file was created, staged, committed, modified, compared against the committed version, and committed again. A deliberately incorrect line was then restored to verify recovery of an uncommitted change. The completed LAB1 materials were maintained in a personal course repository and synchronized with the remote `main` branch.

This procedure distinguished three states clearly: the working tree contains current edits, the staging area defines the next snapshot, and a commit records that snapshot in repository history. Remote synchronization was performed only after the staged file list and repository status had been checked.

4.3 CMake build demonstration

A four-file example project from the course tutorial [3] was recreated locally. Its build definition set the minimum CMake version, declared a C++ project, built a small static library, built a separate executable, and linked the executable to the library. An out-of-source build kept generated build products separate from the source files. The configuration, generation, compilation, and linking stages were all checked before the executable output was recorded.

4.4 C++ warm-up

The warm-up questions covered prefix and postfix increment, the conditional operator, references, pointers, arrays, reference parameters, integer type ranges, floating-point conversion, and signed/unsigned conversion. The answers were determined by following the language operations in execution order and checking the relevant C++ rules against cppreference [2].

4.5 RandomVector implementation

The `RandomVector` class stores floating-point values in a standard vector. Its constructor generates the requested number of values and scales each value to the interval from zero to a specified maximum. Separate member functions traverse the vector to print its contents and calculate the arithmetic mean, minimum, and maximum.

The histogram function first obtains the observed range and divides it into equally sized bins. Each value is mapped to a bin, and the maximum observed value is explicitly assigned to the final bin to avoid an out-of-range index at the upper boundary. The bin counts are printed vertically from the largest count down to one. A fixed seed was used in the test program so that compilation and execution could be checked against the same numerical output.

5 Results

5.1 Shell results

Table 2 shows the measured text statistics. Blank lines explain why the non-empty line count is lower than the total line count.

Table 2: Measured shell-processing results.

Measurement	Result
Total lines	19,567
Total words	97,676
Total bytes	557,042
Non-empty lines	14,338
Collected fortune messages	5

The file-management and pipeline exercises completed without data loss after the difference between overwrite and append redirection had been confirmed.

5.2 CMake demonstration result

The demonstration configured and generated its native build files successfully. Both the library and executable reached 100% build completion. Running the executable printed the two expected lines, `Hello world!` and `In library`, confirming that the executable was linked to and could call the library function.

5.3 C++ warm-up results

Table 3 summarizes the ten required answers. For the unsigned-character question, the specified assumption of an 8-bit character gives conversion modulo 256, so assigning -1 produces 255.

Table 3: C++ warm-up answers.

Topic and question	Result
Operators 1	Final values: $i = 2, j = 1$
Operators 2	Output: <code>b</code>
References and pointers 1	Output: 10 10
References and pointers 2	Output: 1764
References and pointers 3	Output: 0
References and pointers 4	Output: 0
Numbers 1	The types differ in required minimum range and implementation-dependent size; their sizes are non-decreasing from <code>short</code> to <code>long long</code> .
Numbers 2	<code>double</code> normally provides greater precision than <code>float</code> ; assigning 3.14 to an integer gives $i = 3$.
Numbers 3	Signed types represent negative and non-negative values, whereas unsigned types represent non-negative values only; $c = 255$.
Numbers 4	The condition is true for 42, so the assignment in the first branch gives $i = 0$.

5.4 RandomVector results

The implementation compiled without warnings under the selected options and reproduced the saved result. Table 4 reports the primary statistics.

Table 4: RandomVector test summary.

Quantity	Result
Number of generated values	20
Requested maximum value	1.0
Number of histogram bins	5
Mean	0.55804
Minimum	0.0278694
Maximum	0.982655

Table 5 gives the counts represented by the vertical text histogram. The counts sum to 20, which independently confirms that every generated value was assigned to exactly one bin.

Table 5: Five-bin histogram reconstructed from the verified output.

Interval	Count	Relative visual frequency
[0.027869, 0.218827)	5	
[0.218827, 0.409784)	1	
[0.409784, 0.600741)	4	
[0.600741, 0.791698)	3	
[0.791698, 0.982655]	7	

5.5 Version-control result

The required source files and exercise outputs were committed to the personal repository. The local `main` branch and the remote `main` branch resolved to the same commit after synchronization, and the final working tree contained no uncommitted changes. Generated executables, build products, tutorial-only files, and unrelated repositories were excluded from version control.

6 Discussion

The experiment demonstrated why a staged workflow is useful. Inspecting changes before committing prevented unrelated course repositories and generated binaries from entering the submission. It also showed that restoring an uncommitted mistake is different from removing a committed version: the former returns the working file to the latest snapshot without rewriting repository history.

Three practical problems occurred. First, overwrite redirection initially replaced existing text, while append redirection preserved it and added new content. This clarified that redirection choice is part of data safety rather than only command syntax. Second, access from WSL to GitHub was intermittently affected by TLS and connection timeouts. The Windows network stack was therefore used for file transfer while the actual Linux exercises and compilation remained in Ubuntu. Third, a cached GitHub identity did not own the target repository. Clearing that credential and authorizing the correct account resolved the permission failure.

The `RandomVector` program satisfied the assigned test, but it has clear boundaries. The traditional pseudo-random generator is suitable for a deterministic introductory exercise but is not the preferred interface for modern statistical software. The current methods also assume a non-empty vector and a positive bin count. Production code should validate these conditions and should use the C++ random-number library when distribution quality is important.

The course material recommends Ubuntu 20.04, whereas this experiment used Ubuntu 22.04.5 under WSL2. The required LAB1 behaviour was verified successfully in this environment, but later laboratories with hardware drivers or graphical middleware may require additional compatibility checks.

7 Conclusion

LAB1 established a functional Ubuntu-based environment and completed the required Linux shell, CMake, Git, C++ warm-up, and `RandomVector` tasks. The measured shell statistics were recorded, the CMake library and executable built and ran correctly, all ten language questions were answered, the C++ implementation compiled and reproduced its expected statistics and histogram, and the graded materials were synchronized successfully. The resulting workflow provides a practical foundation for later visual-navigation laboratories while also identifying environment and input-validation issues that should be considered as project complexity increases.

References

- [1] MIT Visual Navigation for Autonomous Vehicles, “Lab 1: Linux, C++, and Git,” 2023. https://vnav.mit.edu/labs_2023/lab1/
- [2] cppreference.com, “C++ language reference.” <https://en.cppreference.com/w/cpp/language.html>
- [3] MIT Visual Navigation for Autonomous Vehicles, “Build with CMake,” 2023. https://vnav.mit.edu/labs_2023/lab1/cmake.html